Application to Jerome A. Schiff Fellowship:

Formally Verifying High-Performance Program Optimizations through Scheduling Rewrites

Yuhe Xue
Computer Science ’27
yx105@wellesley.edu

Thesis Co-Advisors:
Professor Adam Chlipala, MIT
Professor Alexa VanHattum, Wellesley College

My Computer Science Honors Thesis explores the formal verification of program optimizations used in modern artificial intelligence (AI) systems. As AI systems demand ever faster and more efficient computation, programs are continually optimized for performance, making it increasingly important to ensure that these optimizations are correct. Many high-performance programs used in practice rely on hand-derived optimizations whose correctness is established primarily through informal mathematical reasoning and extensive testing, approaches that cannot always rule out subtle errors in complex numerical code. Formal verification offers a stronger guarantee: machine-checked mathematical proofs that an optimized program preserves the behavior of its original version.

My thesis targets the formal verification of program optimizations performed through scheduling rewrites, which improve efficiency by reorganizing computations without changing their mathematical results. Specifically, I study scheduling rewrites that collapse computations requiring multiple passes over data into single-pass computations that recursively compute and update intermediate results. A prominent example is FlashAttention, an important optimization of attention, a computation at the core of modern AI models. Attention allows a model to assign different importance to different parts of its input when processing information. For example, in language processing, an attention matrix represents how strongly each token relates to every other token, helping the model determine which information is most relevant when constructing its internal representations and making predictions. FlashAttention transforms attention's multi-pass computation, including the softmax step that normalizes attention scores into a probability distribution, into an online computation that incrementally maintains and updates the intermediate values needed to produce the same result. By doing so, it avoids storing the large intermediate matrix required by standard attention, whose size grows quadratically with the input.

Because FlashAttention can be derived from standard attention through algebraic transformations, scheduling rewrites provide a promising approach to formally verifying its correctness. Its derivation can be represented as a sequence of rewrite rules, each of which transforms the program while preserving its mathematical meaning. However, this derivation introduces recurrences, in which later computations depend on intermediate results computed earlier. State-of-the-art programming languages for scheduling

computation such as Halide and TVM do not support this recurrence-based derivation together with machine-checked proofs of correctness, while existing verified frameworks such as ATL lack support for recurrences. This leaves this class of optimizations beyond the reach of existing verification tools.

My thesis addresses this gap by extending A Tensor Language (ATL) (Liu et al., POPL 2022) with tools for expressing and formally verifying recurrence-based optimizations. ATL is a tensor computation framework developed by the Programming Languages and Verification Group at MIT CSAIL and embedded in Rocq, a proof assistant that mechanically checks mathematical proofs. Structurally, ATL consists of three layers: a shallow embedding, where language constructs and their mathematical properties can be defined and reasoned about; a deep embedding, where these constructs are explicitly represented and given formal semantics; and a formalized subset of the commonly used C programming language, to which programs in the deep embedding can ultimately be lowered.

ATL is particularly suitable for my project because it already supports transformations such as tiling and loop fusion that are also necessary for the class of optimizations I target. I will build on these existing features to design constructs that support dependent recurrence computations within fused loops, extend ATL's library of verified algebraic rewrite rules, and extend its compilation infrastructure to support the new language constructs. I will use FlashAttention as my primary case study. Specifically, I will use my new language constructs to represent FlashAttention's structure as a fused loop of dependent recurrence arrays, combine my new rewrite rules with ATL's existing ones to derive FlashAttention from standard attention, and show that the resulting program can be correctly lowered to ATL's formalized subset of C. Because ATL is embedded in Rocq, each definition, rewrite rule, and transformation can be accompanied by a machine-checked proof, allowing the complete derivation to establish an end-to-end guarantee that the optimized program preserves the mathematical meaning of the original. Together, these extensions will enable ATL to formally verify a broader class of program optimizations that introduce recurrences during transformation.

I will work with two co-advisors: Professor Adam Chlipala, principal investigator of the Programming Languages and Verification Group at MIT CSAIL, where ATL was developed and where I

worked as an undergraduate researcher through MIT's UROP program during Summer 2026; and Professor Alexa VanHattum of the Wellesley College Computer Science Department, who also advised my independent study last spring on lightweight verification of instruction selection in Cranelift, a production WebAssembly compiler.

I have already made substantial progress toward these goals. I designed and implemented a core construct in ATL's shallow embedding, WideLetRec, that supports a fused loop of dependent recurrence arrays, and developed a prototype schedule of rewrites that derives FlashAttention from standard attention. I am currently decomposing these transformations into smaller, more precisely defined rewrite rules and proving their correctness so that they capture general algebraic properties rather than assumptions specific to FlashAttention. Designing the rules at this level of generality will allow them to be reused in other rewrite schedules for recurrence-based optimizations.

For the rest of Fall 2026, I plan to complete one major component each month while continuing to develop and verify the rewrite rules used in FlashAttention's derivation. First, I will implement WideLetRec in ATL's deep embedding and define and prove the evaluation rules that establish its semantics. Second, I will implement and prove the rules for lowering WideLetRec to ATL's formalized subset of C. Third, I will implement the reification procedure, which translates WideLetRec from the shallow embedding to the deep embedding. I have structured these components so that each builds toward a self-contained research result: even if the lowering proofs take longer than anticipated, the verified rewrite rules and FlashAttention derivation will stand as a complete contribution. During Spring 2027, I will complete remaining implementation and proof development and focus primarily on writing the thesis.

Formal proofs require repeatedly refining program definitions and the mathematical statements needed to establish their correctness. Fellowship support would allow me to devote sustained research time, including during Wintersession, to completing the implementation and verification before focusing on writing in the spring. The fellowship would also allow me to participate in academic venues such as the New Jersey Programming Languages and Systems Seminar (NJPLS), giving me greater exposure to the programming languages community and current research while I work toward completing my thesis.